

利用蒙特卡洛方法模拟隔离政策对疫情传播的影响

231830128 茅单文

June 28, 2025

1 引言

传染病动力学建模对于理解疾病传播模式及评估公共卫生干预措施的有效性至关重要。传统的房室模型（如 SIR 模型）在宏观层面描述了不同状态人群的转化，但难以刻画个体异质性、空间交互等微观因素。蒙特卡洛方法，特别是基于智能体（Agent-Based Model, ABM）的模拟，为此提供了有力的解决方案。通过为系统中的每个个体（智能体）赋予独立的属性和随机性行为规则，ABM 能够从微观的个体互动中涌现出宏观的群体传播现象。

本研究构建了一个基于智能体的传染病传播模型，运用蒙特卡洛模拟方法，旨在探究隔离政策对疫情传播过程的动态影响。通过对比“有隔离”与“无隔离”两种情景下的疫情关键指标（如感染峰值、累计感染与死亡人数），本报告旨在量化分析隔离措施在“压平曲线”和降低总体健康损失方面的作用。

2 蒙特卡洛模拟方法

模拟的核心是构建一个包含多个随机过程的计算模型，通过大量随机抽样来模拟真实世界的不确定性。

2.1 智能体与环境

模型设定在一个 $L \times L$ 的二维连续空间中，容纳 N_{pop} 个智能体。每个智能体在模拟过程中始终处于以下四种状态之一：

- 易感者 (Susceptible, S)，未免疫的活人；
- 感染者 (Infected, I)，病人；
- 免疫者 (Immune, R)，三次感染并痊愈的个体。
- 死亡者 (Deceased, D)，死人。

2.2 核心随机过程建模

疫情的传播被分解为个体层面的多个随机事件，其具体规则如下：

1. **个体移动（随机游走）**：个体的空间移动是接触和传播的前提。在每个时间步，非死亡个体的移动由随机抽样决定。移动距离 d 和方向改变量 $\Delta\theta$ 分别从均匀分布 $U(0, v_{move})$ 和 $U(-\pi/4, \pi/4)$ 中抽取。这种随机游走过程是典型的马尔可夫链蒙特卡洛（MCMC）思想的应用。
2. **疾病传播（概率事件）**：当一个易感者与一个感染者的空间距离小于传染半径 R_{inf} 时，触发一次潜在的传播事件。是否成功感染由一次伯努利试验决定，其成功概率为有效感染率 $P_{eff} = P_{inf} \times M_{susc}(n_{inf})$ ，其中 P_{inf} 是基础感染率，而 M_{susc} 是一个依赖于个体历史感染次数 n_{inf} 的易感性折扣因子。这种设计体现了个体异质性。
3. **疾病进程（随机时长与结果）**：感染者的病程同样是随机的。每个新感染者的恢复倒计时 T_{rec} 从整数均匀分布 $U(T_{min}, T_{max})$ 中随机抽取，病程结束后，通过一次随机抽样决定其结局。以病死率 P_{fatal} 的概率死亡，否则康复。这又是一次伯努利试验。值得注意的是，当医疗系统过载时，病死率将切换至更高的 P'_{fatal} 。

3 模拟情景设定

为评估隔离政策的影响，本研究设计了两种并行的模拟情景。

3.1 基准情景：无干预

在此情景下，所有智能体仅遵循第二节中描述的基本随机过程进行移动、互动和状态演变，不施加任何外部干预。此情景作为评估政策效果的对照组。

3.2 干预情景：隔离政策

此情景引入了一套基于检测的动态隔离措施：当现存感染者占总人口比例达到阈值 Q_{thr} 时，隔离政策被激活，此后每隔 T_{test} 天进行全员检测。检测过程本身包含不确定性，通过蒙特卡洛方法模拟：对实际感染者，有 R_{fn} 的概率得到假阴性结果；而对非感染者，有 R_{fp} 的概率得到假阳性结果。这种不完美的检测机制是模型现实性的重要体现。隔离期间，所有个体的移动速度均受限，而其受限程度根据由最近一次的检测结果决定。

4 模型参数

模拟中使用的主要参数及其取值如下表所示。

Table 1: 模型关键参数定义与取值		
参数	符号	值/描述
基本参数		
总人口	N_{pop}	1000
区域边长	L	100
初始感染人数	I_0	5
模拟总天数	T_{sim}	400
基础感染概率	P_{inf}	0.1
传染半径	R_{inf}	3
最短恢复时间	T_{min}	14 天
最长恢复时间	T_{max}	21 天
基础病死率	P_{fatal}	0.02
正常移动速度	v_{move}	3
干预策略相关参数		
隔离触发阈值	Q_{thr}	20%
隔离期健康者速度	$v_{q,h}$	0.8
隔离期感染者速度	$v_{q,i}$	0.5
检测周期	T_{test}	7 天
假阳性率	R_{fp}	0.05
假阴性率	R_{fn}	0.10
医疗系统相关参数		
医疗承载阈值	H_{cap}	200 人
挤兑下病死率	P'_{fatal}	0.05

5 一次典型结果

5.1 结果描述

从图中可以清晰地观察到两种情景下疫情发展轨迹的显著差异：

无隔离政策情景下，当前感染人数呈现出急剧爆发的特征，曲线在极短时间内快速攀升，约在第 55 天达到接近 750 人的峰值，随后迅速回落，整个爆发周期较为集中和短暂。累计感染人数呈典型的 S 型增长，增长速度极快，在约 150 天后趋于平稳，最终达到约 2400 人次的高位平台。累计死亡人数与感染曲线同步快速增长，最终稳定在约 125 人。

有隔离政策情景下，隔离政策于第 23 天启动，此后疫情发展曲线开始偏离无政策情景，当前感染人数增长速度明显放缓，其峰值被显著压平，约在第 75 天达到约 350 人的峰值，远低于无政策情景；同时，疫情的持续时间被“拉长”，曲线下降更为平缓。累计感染人数的增长斜率更为平缓，最终稳定在约 1700 人次的平台，显著低于无政策情景。累计死亡人数的增长也更为缓慢，最终稳定在约 55 人，不到无政策情景的一半。

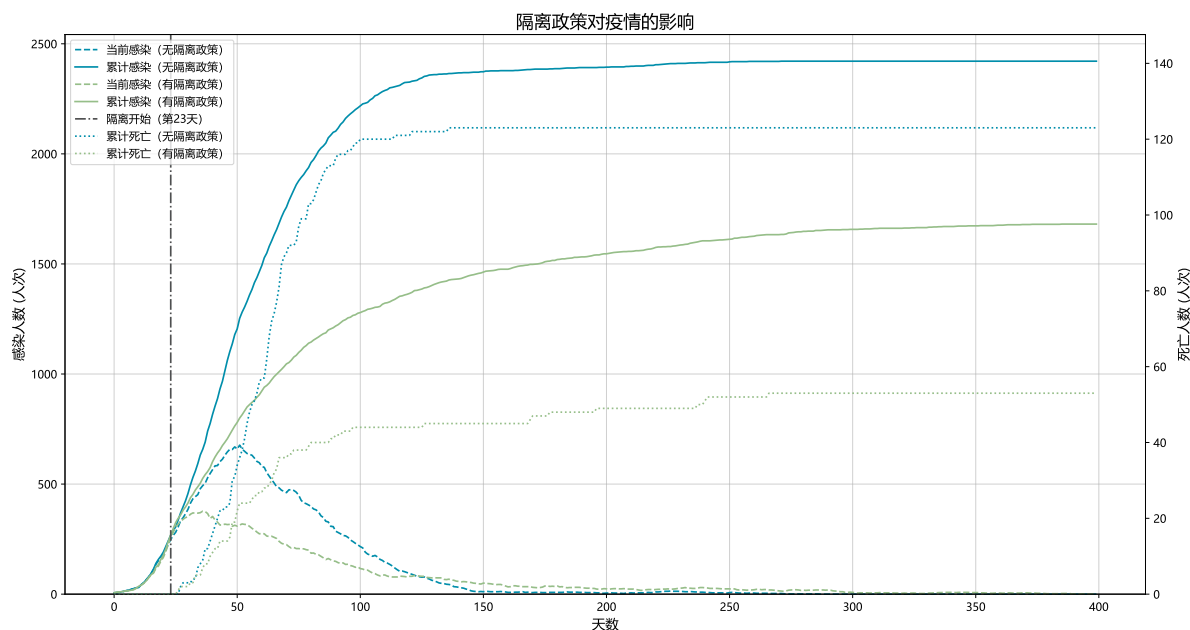


Figure 1: 疫情关键指标随时间变化的一次典型结果

5.2 机理分析

模拟结果直观地展示了隔离政策在控制疫情方面的多重积极作用，其核心机制在于通过限制个体移动来降低接触频率，从而打断病毒的传播链。其具体作用包括：

压平曲线：隔离政策最显著的成效是大幅降低了感染峰值。在无干预时，病毒的指数级传播导致短期内出现大量感染者，这极易造成医疗系统的挤兑。一旦当前感染人数超过医疗承载能力，病死率会上升，导致更多死亡。隔离政策通过减缓传播速度，将感染人数的峰值控制在更低的水平，有效缓解了医疗系统的瞬时压力，这是死亡人数大幅下降的关键原因之一。

降低总感染规模：通过早期限制接触，隔离政策不仅推迟了感染高峰的到来，更重要的是保护了大部分易感人群，使得疫情在蔓延到整个群体之前就得到了有效遏制。这直接体现在最终累计感染人数的显著降低上（从约 2400 人次降至 1700 人次）。

减少总体健康损失：累计死亡人数的显著下降是上述两点综合作用的结果。一方面，总感染人数减少，直接导致面临死亡风险的基数变小；另一方面，“压平曲线”避免了医疗挤兑下的高病死率，提高了患者的生存机会。

综上所述，蒙特卡洛模拟结果有力地证明，尽管隔离政策可能会延长疫情的持续时间，但它通过“压平曲线”和降低总感染规模，能够极其有效地减少疫情所造成的总体健康损失，特别是降低死亡人数，是应对大规模传染病爆发的必要且有效的公共卫生干预措施。

6 完整代码

```
import random
import numpy as np
import math
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.ticker import FuncFormatter

POPULATION_SIZE = 1000 # 总人数
GRID_SIZE = 100 # 区域边长
INITIAL_INFECTED = 5 # 初始感染人数
SIMULATION_DAYS = 400 # 模拟时长
INFECTION_PROB = 0.1 # 感染率
INFECTION_RADIUS = 3 # 传染半径
RECOVERY_TIME_MIN = 14 # 最短痊愈时间
```

```

RECOVERY_TIME_MAX = 21 # 最长痊愈时间
FATALITY_RATE = 0.02 # 病死率
MOVEMENT_SPEED = 3 # 正常移动速度
QUARANTINE_THRESHOLD = 0.2 # 启动隔离时的感染者比例
QUARANTINE_HEALTHY_SPEED = 0.8 # 隔离状态下, 未确诊者移动速度
QUARANTINE_INFECTED_SPEED = 0.5 # 隔离状态下, 确诊者移动速度
TESTING_INTERVAL = 7 # 诊断周期
FALSE_POSITIVE_RATE = 0.05 # 假阳性率
FALSE_NEGATIVE_RATE = 0.10 # 假阴性率
HEALTHCARE_CAPACITY_THRESHOLD = 200 # 医疗系统可容纳的总人数
STRAINED_FATALITY_RATE = 0.05 # 医疗挤兑下的病死率
SUSCEPTIBLE = 0 # 健康者
INFECTED = 1 # 病患
IMMUNE = 2 # 免疫者 (已感染三次并痊愈)
DECEASED = 3 # 死者

class Person:
    def __init__(self, id, x, y, state=SUSCEPTIBLE):
        self.id = id; self.x = x; self.y = y; self.state = state
        self.infected_day = -1; self.recovery_countdown = -1
        self.direction_angle = random.uniform(0, 2 * math.pi)
        self.infection_count = 0
        self.perceived_state = state

    def get_susceptibility_multiplier(self):
        if self.state != SUSCEPTIBLE: return 0.0
        if self.infection_count == 0: return 1.0
        elif self.infection_count == 1: return 0.6
        elif self.infection_count == 2: return 0.3
        return 0.0

    def move(self, quarantine_active=False):
        if self.state == DECEASED: return
        if quarantine_active:
            speed = QUARANTINE_INFECTED_SPEED if self.perceived_state == INFECTED else QUARANTINE_HEALTHY_SPEED
            angle = random.uniform(0, 2 * math.pi); distance = random.uniform(0, speed)
            dx, dy = distance * math.cos(angle), distance * math.sin(angle)
            self.x = max(0, min(GRID_SIZE, self.x + dx)); self.y = max(0, min(GRID_SIZE, self.y + dy))
        else:
            move_angle = self.direction_angle + random.uniform(-math.pi / 4, math.pi / 4)
            distance = random.uniform(0, MOVEMENT_SPEED)
            dx, dy = distance * math.cos(move_angle), distance * math.sin(move_angle)
            self.x = (self.x + dx) % GRID_SIZE; self.y = (self.y + dy) % GRID_SIZE

    def update_state(self, current_day, healthcare_strained=False):
        if self.state == INFECTED and current_day - self.infected_day >= self.recovery_countdown:
            current_fatality_rate = STRAINED_FATALITY_RATE if healthcare_strained else FATALITY_RATE
            if random.random() < current_fatality_rate:
                self.state = DECEASED
            else:
                self.infection_count += 1
                self.state = IMMUNE if self.infection_count >= 3 else SUSCEPTIBLE

            self.perceived_state = self.state
            return True
        return False

def initialize_population():

```

```

population = []
for i in range(POPULATION_SIZE):
    x, y = random.uniform(0, GRID_SIZE), random.uniform(0, GRID_SIZE)
    population.append(Person(i, x, y))
infected_indices = random.sample(range(POPULATION_SIZE), INITIAL_INFECTED)
for idx in infected_indices:
    p = population[idx]
    p.state = INFECTED
    p.perceived_state = INFECTED
    p.infected_day = 0
    p.recovery_countdown = random.randint(RECOVERY_TIME_MIN, RECOVERY_TIME_MAX)
return population

def simulate_day(population, current_day, quarantine_active=False, healthcare_strained=False):
    for person in population:
        person.move(quarantine_active=quarantine_active)
    potential_targets = [p for p in population if p.state == SUSCEPTIBLE]
    infectious_people = [p for p in population if p.state == INFECTED]
    newly_infected = set()
    for s_person in potential_targets:
        for i_person in infectious_people:
            if math.sqrt((s_person.x - i_person.x)**2 + (s_person.y - i_person.y)**2) < INFECTION_RA:
                effective_prob = INFECTION_PROB * s_person.get_susceptibility_multiplier()
                if random.random() < effective_prob:
                    newly_infected.add(s_person); break
    for person in newly_infected:
        person.state = INFECTED; person.infected_day = current_day
        person.recovery_countdown = random.randint(RECOVERY_TIME_MIN, RECOVERY_TIME_MAX)
        if quarantine_active:
            person.perceived_state = SUSCEPTIBLE

    for person in population:
        person.update_state(current_day, healthcare_strained=healthcare_strained)
    return len(newly_infected)

def run_simulation(quarantine_enabled=False):
    population = initialize_population()
    s_hist, i_hist, r_hist, d_hist, cumulative_hist = [], [], [], [], []
    animation_frames = []; quarantine_triggered = False; quarantine_day = -1
    total_infection_events = INITIAL_INFECTED
    for day in range(SIMULATION_DAYS):
        i_count = sum(1 for p in population if p.state == INFECTED)
        healthcare_strained = i_count > HEALTHCARE_CAPACITY_THRESHOLD
        if quarantine_enabled:
            if not quarantine_triggered and (i_count / POPULATION_SIZE) >= QUARANTINE_THRESHOLD:
                quarantine_triggered = True; quarantine_day = day

        if quarantine_triggered and day > 0 and day % TESTING_INTERVAL == 0:
            for p in population:
                if p.state == DECEASED: continue
                if p.state == INFECTED:
                    if random.random() < FALSE_NEGATIVE_RATE:
                        p.perceived_state = SUSCEPTIBLE
                    else:
                        p.perceived_state = INFECTED
                else:
                    if random.random() < FALSE_POSITIVE_RATE:
                        p.perceived_state = INFECTED

```

```

        else:
            p.perceived_state = p.state

    num_new_infections = simulate_day(population, day, quarantine_active=quarantine_triggered, h
    total_infection_events += num_new_infections
    s_count = sum(1 for p in population if p.state == SUSCEPTIBLE)
    i_count = sum(1 for p in population if p.state == INFECTED)
    r_count = sum(1 for p in population if p.state == IMMUNE)
    d_count = sum(1 for p in population if p.state == DECEASED)
    s_hist.append(s_count)
    i_hist.append(i_count)
    r_hist.append(r_count)
    d_hist.append(d_count)
    cumulative_hist.append(total_infection_events)
    animation_frames.append([(p.x, p.y, p.state) for p in population])
    return s_hist, i_hist, r_hist, d_hist, cumulative_hist, animation_frames, quarantine_day

def plot_combined_report(hists1, hists2, quarantine_day):
    s_hist1, i_hist1, r_hist1, d_hist1, cumulative_hist1 = hists1
    s_hist2, i_hist2, r_hist2, d_hist2, cumulative_hist2 = hists2
    days1 = np.arange(len(i_hist1)); days2 = np.arange(len(i_hist2))
    current_infected1 = np.array(i_hist1); cumulative_deceased1 = np.array(d_hist1)
    current_infected2 = np.array(i_hist2); cumulative_deceased2 = np.array(d_hist2)
    fig, ax1 = plt.subplots(figsize=(15, 8)); ax2 = ax1.twinx()
    ax1.plot(days1, current_infected1, color=(0/255, 142/255, 171/255), linestyle='--', label='当前感
    ax1.plot(days1, cumulative_hist1, color=(0/255, 142/255, 171/255), linestyle='-', label='累计感
    ax2.plot(days1, cumulative_deceased1, color=(0/255, 142/255, 171/255), linestyle=':', label='累
    ax1.plot(days2, current_infected2, color=(150/255, 190/255, 137/255), linestyle='--', label='当前
    ax1.plot(days2, cumulative_hist2, color=(150/255, 190/255, 137/255), linestyle='-', label='累计感
    ax2.plot(days2, cumulative_deceased2, color=(150/255, 190/255, 137/255), linestyle=':', label='累
    if quarantine_day != -1: ax1.axvline(x=quarantine_day, color=(78/255, 79/255, 80/255), linestyle
    ax1.set_xlabel('天数', fontsize=12); ax1.set_ylabel('感染人数 (人次)', fontsize=12)
    ax2.set_ylabel('死亡人数 (人次)', fontsize=12)
    ax1.set_title('隔离政策对疫情的影响', fontsize=16)
    ax1.set_ylim(bottom=0); ax2.set_ylim(bottom=0)
    max_deaths = max(max(cumulative_deceased1), max(cumulative_deceased2)) if len(cumulative_decease
    ax2.set_ylim(top=max_deaths * 1.2 if max_deaths > 0 else 10)
    lines1, labels1 = ax1.get_legend_handles_labels(); lines2, labels2 = ax2.get_legend_handles_labe
    ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left', fontsize=10); ax1.grid(True, al

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei']
plt.rcParams['axes.unicode_minus'] = False
hists1_data = run_simulation(quarantine_enabled=False)
hists2_data = run_simulation(quarantine_enabled=True)
plot_combined_report(hists1_data[:5], hists2_data[:5], hists2_data[6])
anim_frames1 = hists1_data[5]; anim_frames2 = hists2_data[5]
fig2, (anim_ax1, anim_ax2) = plt.subplots(1, 2, figsize=(12, 6), sharey=True, sharex=True)
fig2.suptitle('疫情传播模拟', fontsize=16)
for ax, title in zip([anim_ax1, anim_ax2], ['无隔离政策', '有隔离政策']):
    ax.set_xlim(0, GRID_SIZE); ax.set_ylim(0, GRID_SIZE); ax.set_aspect('equal', adjustable='box')
    ax.set_xticks([]); ax.set_yticks([]); ax.set_title(title, fontsize=14)
    scatter1 = anim_ax1.scatter([], [], s=10); scatter2 = anim_ax2.scatter([], [], s=10)
    color_map = {SUSCEPTIBLE: 'blue', INFECTED: 'red', IMMUNE: 'green', DECEASED: 'black'}
    day_text1 = anim_ax1.text(0.02, 1.02, '', transform=anim_ax1.transAxes)
    day_text2 = anim_ax2.text(0.02, 1.02, '', transform=anim_ax2.transAxes)
def update_animation(day):
    if day < len(anim_frames1):
        frame_data1 = anim_frames1[day]

```

```

scatter1.set_offsets(np.c_[[p[0] for p in frame_data1], [p[1] for p in frame_data1]])
scatter1.set_color([color_map[p[2]] for p in frame_data1])
day_text1.set_text(f'第{day}天')
if day < len(anim_frames2):
    frame_data2 = anim_frames2[day]
    scatter2.set_offsets(np.c_[[p[0] for p in frame_data2], [p[1] for p in frame_data2]])
    scatter2.set_color([color_map[p[2]] for p in frame_data2])
    #day_text2.set_text(f'第 {day} 天')
return scatter1, scatter2, day_text1, day_text2
num_frames = max(len(anim_frames1), len(anim_frames2))
ani = animation.FuncAnimation(fig2, update_animation, frames=num_frames, blit=False, interval=50)
plt.show()

```